부록 · 파이썬으로 DB 다루기

값은 %s 자리로, 비밀번호는 .env 로

코드가 돌아가는 것만큼 중요한 두 가지 습관입니다. 실제로 막아지는 것을 눈으로 확인합니다.

왜 문자열로 이어 붙이면 안 되나 — SQL 인젝션

검색어를 f-string 으로 SQL 에 바로 이어 붙이면, 검색어 안의 따옴표가 SQL 의 일부가 됩니다. 아래 코드는 누군가 검색창에 「' OR '1'='1」 을 넣은 경우입니다.

.env — 비밀번호를 코드 밖으로

코드와 같은 폴더에 .env 파일을 만들고 접속 정보를 적습니다. 코드에는 비밀번호가 남지 않습니다.

```env
DB_HOST=127.0.0.1
DB_PORT=3306
DB_USER=root
DB_PASSWORD=dataeng123
DB_NAME=shop
```

```bash
# .env 는 깃에 올리지 않습니다 — .gitignore 에 한 줄
echo ".env" >> .gitignore
```

코드

```python
# py_mysql_safe.py — 값은 %s 자리로, 비밀번호는 .env 로
import os
import pymysql
from dotenv import load_dotenv

load_dotenv()      # 같은 폴더의 .env 를 읽어 환경변수로 올린다
conn = pymysql.connect(host=os.getenv('DB_HOST', '127.0.0.1'), port=int(os.getenv('DB_PORT',
'3306')),
                       user=os.getenv('DB_USER'), password=os.getenv('DB_PASSWORD'),
                       database=os.getenv('DB_NAME'), charset='utf8mb4')

keyword = "' OR '1'='1"     # 검색창에 누군가 이렇게 넣었다고 하자

with conn:
    with conn.cursor() as cur:
        # 나쁜 예 — 문자열로 이어 붙이면 조건이 무너져 전부 나온다
        cur.execute(f"SELECT name FROM products WHERE name = '{keyword}'")
        print('이어 붙인 경우:', len(cur.fetchall()), '행')

        # 좋은 예 — %s 자리로 넘기면 값 전체를 글자 하나로 다룬다
        cur.execute("SELECT name FROM products WHERE name = %s", (keyword,))
        print('%s 로 넘긴 경우:', len(cur.fetchall()), '행')
```

```bash
python3 py_mysql_safe.py
```

실행 결과 (실측)

```text
이어 붙인 경우: 5 행
%s 로 넘긴 경우: 0 행
```

이어 붙인 쪽은 WHERE name = '' OR '1'='1' 이 되어 조건이 항상 참이 됩니다. 그래서 상품 5개가 전부 나왔습니다. %s 로 넘긴 쪽은 그 글자 그대로의 이름을 찾았고, 그런 상품은 없으니 0 행입니다.

짚을 점

- %s 는 PyMySQL 의 자리 표시입니다. 숫자여도 %d 가 아니라 %s 를 씁니다. 값은 언제나 두 번째 인자(튜플)로 넘깁니다.
- 값이 하나일 때도 튜플로 씁니다 — (keyword,) 처럼 쉼표가 있어야 튜플입니다.
- 표 이름·칸 이름은 %s 로 넘길 수 없습니다(값 자리 전용). 이름을 사용자 입력으로 바꿔야 한다면 허용 목록에서 고르게 합니다.
- 수업처럼 root 로 접속하는 것은 실습용입니다. 실제 서비스는 필요한 권한만 준 계정을 따로 만들어 씁니다(부록 「DB 관리 명령어」의 사용자 만들기).
